IEE575 - Applied Stochastic Operations Research Models

Week 4: Bayesian Optimization

Tanmay Khandait
tkhandai@asu.edu

April 28, 2026

Today's Plan

Part 1 — Real-World Demo (30 min)

- The problem: tuning a PID controller
- The system and the cost function
- Why manual tuning fails
- BO finds the answer in 33 evaluations
- What the GP learned about the problem

Part 2 — Lab 7 (remainder)

- Himmelblau: implement & visualize
- Derive and code Expected Improvement
- Run the full BO loop
- Diagnose failure modes
- High-dimensional scaling experiment

Core message: BO is not a theoretical curiosity. It is the standard tool for optimizing expensive black-box functions in engineering, machine learning, and scientific computing.

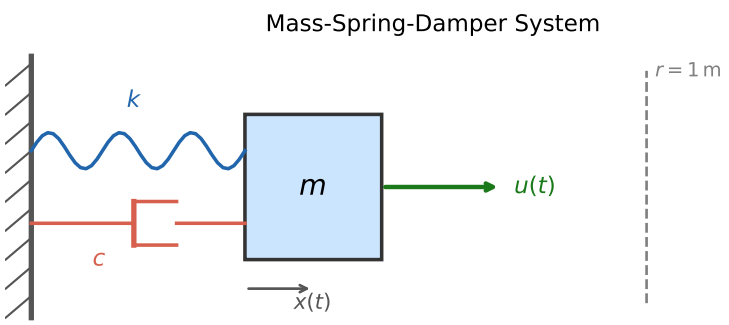
The Problem: Tuning a PID Controller

We want to move a mass to a **target position** $r = 1\,\mathrm{m}$ and hold it there.

The system is a **mass–spring–damper**:

$$m\ddot{x} + c\dot{x} + kx = u(t)$$

Symbol	Meaning	Value
m	mass	$1\,\mathrm{kg}$
c	damping	$0.5\,\mathrm{N\,s/m}$
k	spring const.	$1\,\mathrm{N/m}$
$u(t)$	control force	*we choose*



The PID Controller

The controller computes a force based on the **error** $e(t) = r - x(t)$:

$$u(t) = \underbrace{K_p\, e(t)}_{\text{Proportional}} + \underbrace{K_i \int_0^t e(\tau)\, d\tau}_{\text{Integral}} + \underbrace{K_d\, \dot{e}(t)}_{\text{Derivative}}$$

K_p — Proportional

Push harder the further away you are. Too large $\Rightarrow$ oscillation.

K_i — Integral

Correct persistent steady-state error. Too large $\Rightarrow$ wind-up, instability.

K_d — Derivative

Brake before overshooting. Too large $\Rightarrow$ sluggish response.

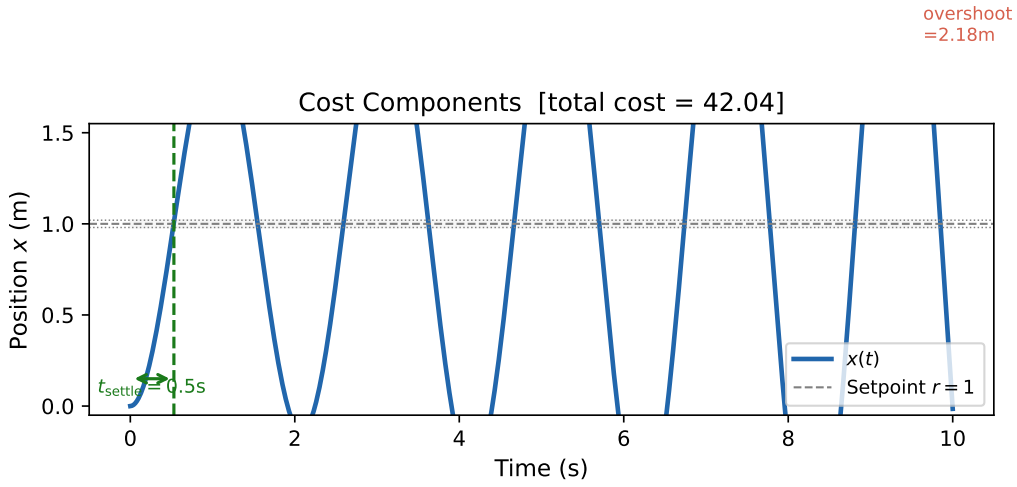
The three gains **interact nonlinearly**. A change in K_p shifts the optimal K_d. Gradient information is unavailable. This is a black-box optimization problem.

The Cost Function

Every set of gains (K_p, K_i, K_d) produces a trajectory $x(t)$. We measure quality with a single scalar cost — **lower is better**:

$$\mathcal{C}(K_p, K_i, K_d) = \underbrace{w_1\, t_{\text{settle}}}_{\text{speed}} + \underbrace{w_2\, \max(x - r,\, 0)}_{\text{overshoot}} + \underbrace{w_3 \int u^2\, dt}_{\text{effort}}$$

with $w_1 = 1,\ w_2 = 2,\ w_3 = 0.1$.

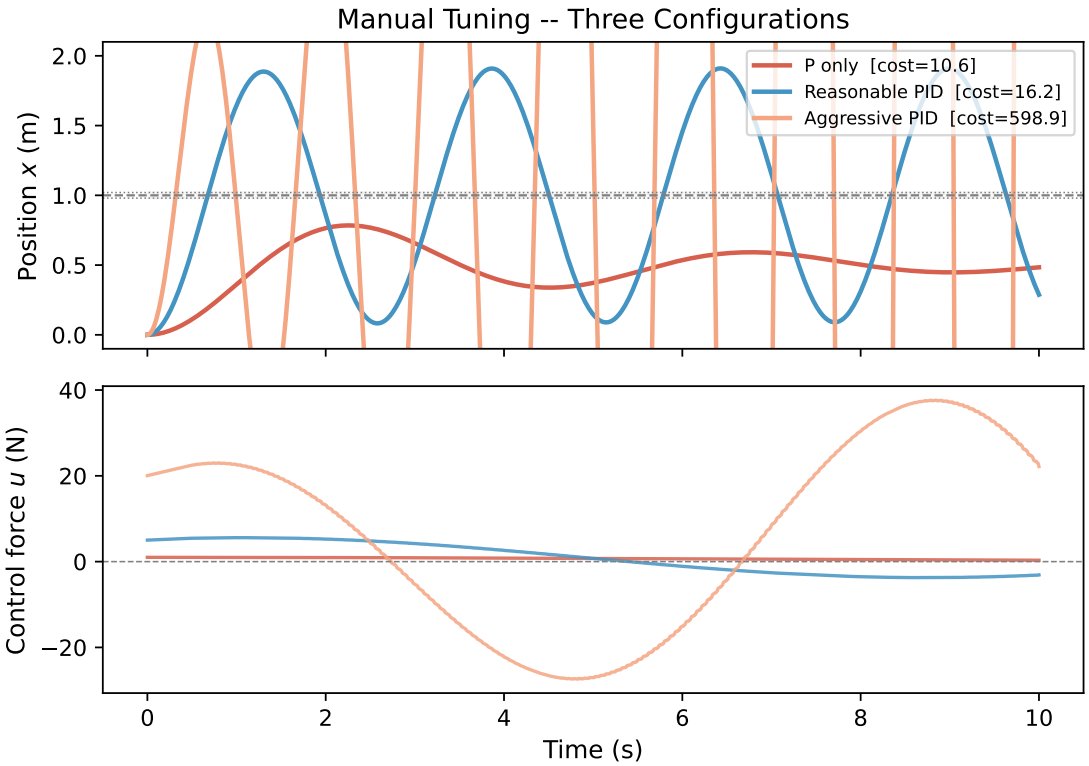


Three penalties in tension:

- Settling fast requires large $u \Rightarrow$ high effort
- Reducing overshoot requires careful K_d braking
- Minimising effort slows the response

The weighting reflects *engineering priorities* — overshoot is penalised twice as heavily as settling time because it can cause physical damage in real systems.

Why Manual Tuning Fails



Three configurations:

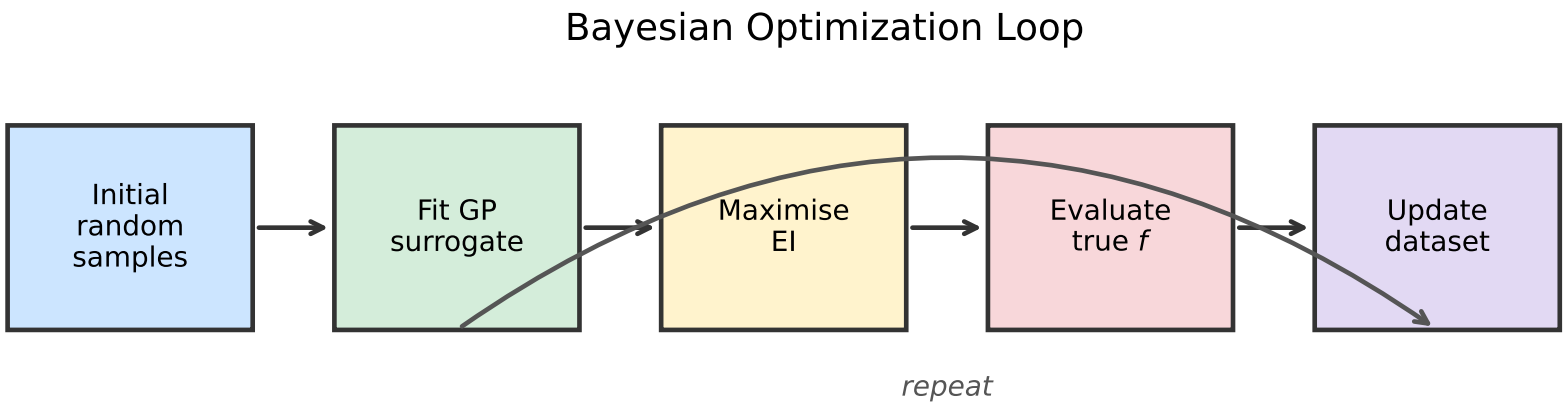
P only
Steady-state error: the spring pulls back and $K_i = 0$ cannot correct it.

Reasonable PID
Reaches the setpoint but oscillates. Requires human iteration to improve.

Aggressive PID
Fast initial response, large overshoot, excessive force. High cost.

Each configuration required an *evaluation of the simulator*. With 3 gains $\times$ 10 levels each $= 1000$ grid points, hand-search is expensive and misses the optimum.

Bayesian Optimization: The Loop



What the GP surrogate gives us:

- $\mu(\boldsymbol{x})$ — predicted cost at any gain triplet
- $\sigma(\boldsymbol{x})$ — uncertainty about that prediction

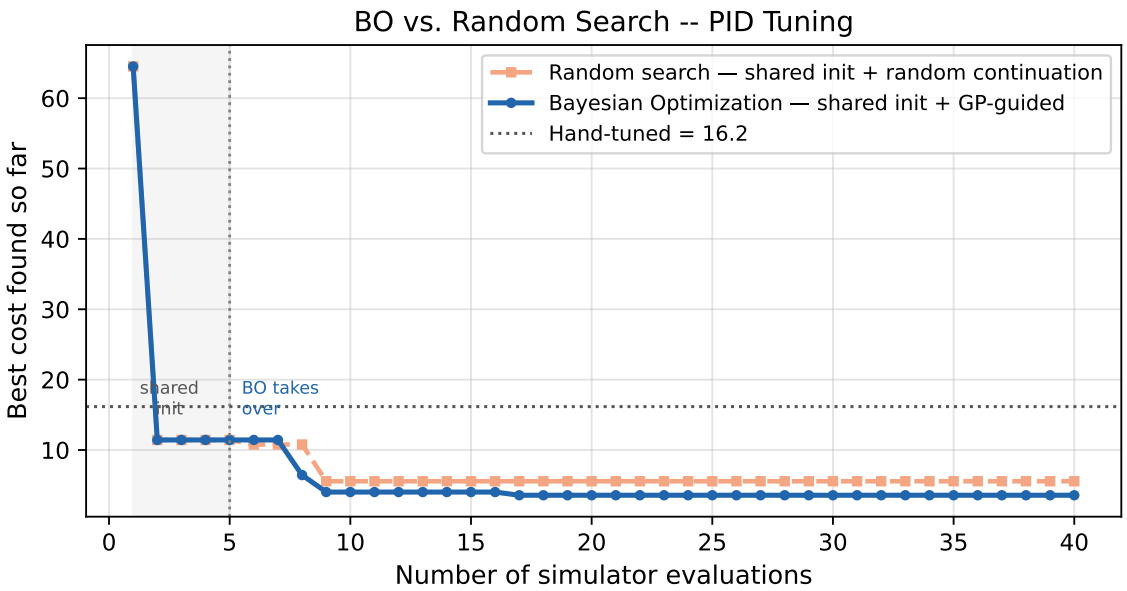
What EI does with it:

$$\mathrm{EI}(\boldsymbol{x}) = (\mu - f^* - \xi)\,\Phi(Z) + \sigma\,\phi(Z)$$

Balances exploiting good regions vs. exploring uncertain ones.

We run **5 shared random initial evaluations** then **35 BO iterations** = 40 total simulator calls. The random search baseline uses the *same 5 starting points*, then draws 35 more randomly — isolating the surrogate as the only variable. Each call takes $\approx 0.1\,\mathrm{s}$. In a real system (e.g., wind tunnel test), each call could take hours.

Results: Convergence



A controlled comparison:

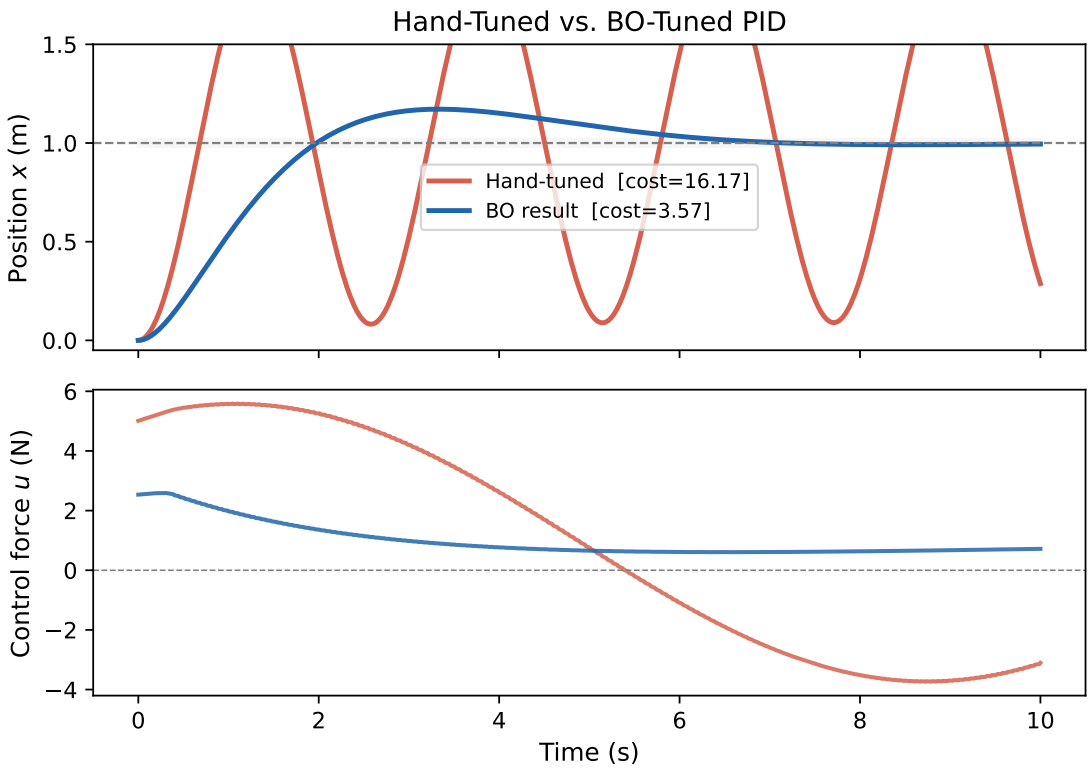
Both strategies start from the **same 5 random points** (grey shaded region). After that, random search keeps sampling blindly; BO uses the GP surrogate to decide where to look next.

What to observe:

- The two curves are *identical* during shared init
- BO diverges sharply downward once the surrogate kicks in
- Random search plateaus; BO keeps improving
- BO reaches a cost of ≈ 3.6 vs. 5.6 for random and 16.2 for hand-tuned

Same starting point, same budget. The only difference is whether you use the information you have.

Results: Best Response



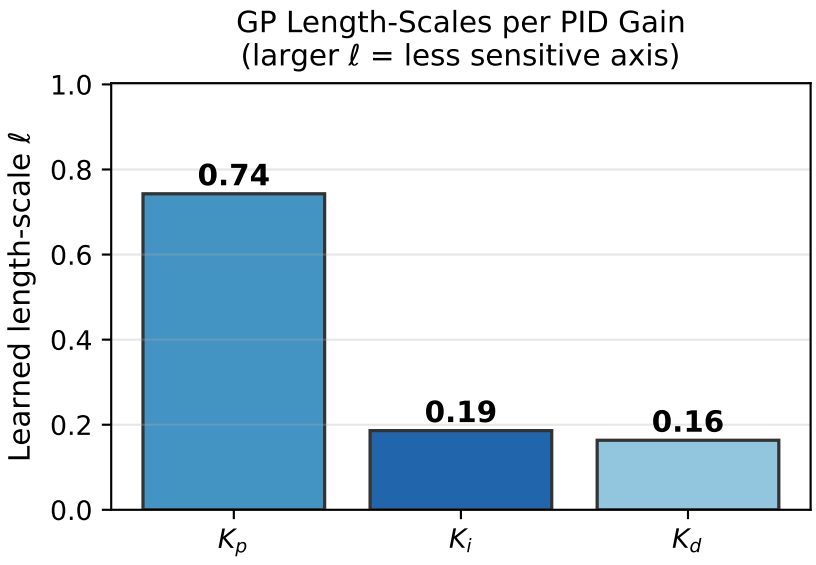
BO-tuned gains:

- Found by optimizing a GP surrogate
- No domain knowledge used
- Faster settling, lower overshoot, less control effort

Control force panel:
The BO controller applies a smooth, controlled force profile. The hand-tuned controller demands larger peak forces and oscillates unnecessarily, wasting energy.

In hardware, lower $\int u^2\, dt$ means less wear on actuators and lower energy consumption — both economically meaningful.

What the GP Learned: Automatic Relevance Determination



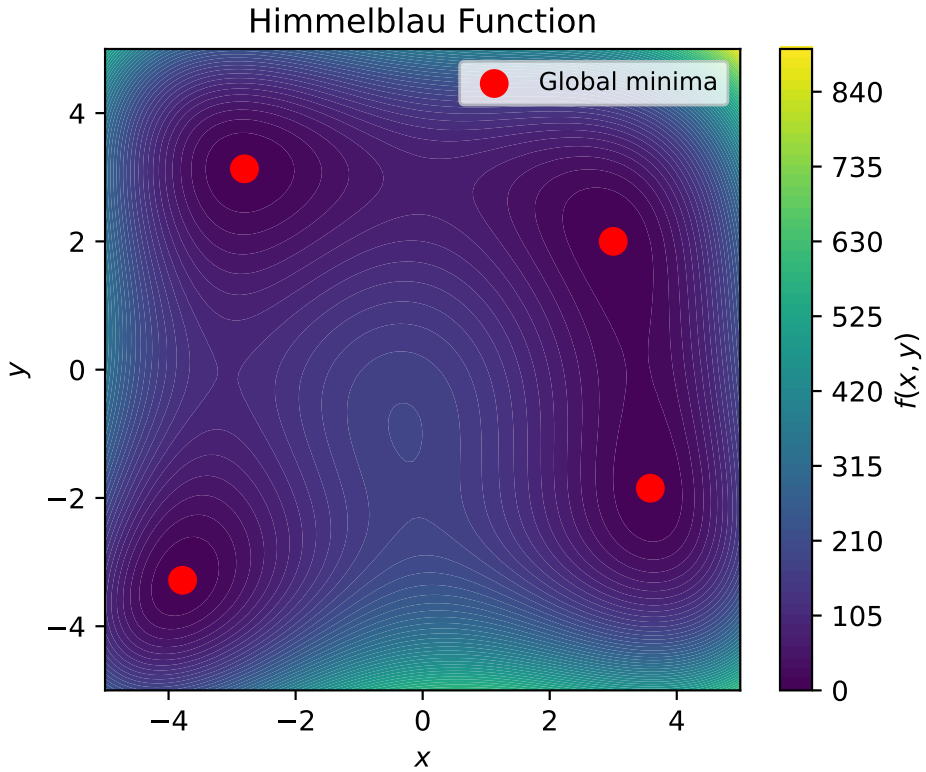
The RBF kernel has one length-scale per dimension:

$$k(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\sum_j \frac{(x_j - x_j')^2}{2\ell_j^2}\right)$$

A **short** ℓ_j means the cost changes rapidly along axis j — small changes to that gain matter a lot.
A **long** ℓ_j means the cost is relatively flat — that gain is less critical to get exactly right.

This is **Automatic Relevance Determination (ARD)**. The GP discovers which inputs matter *without being told*. SAASBO (Lab 3, Section 5) extends this idea to 50–1000 dimensions using a sparsity prior on ℓ_j.

Lab 3 Overview



Objective function: Himmelblau

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

Four global minima, all at $f = 0$. Multimodal, non-convex, smooth.

Five sections:

1. Implement & visualize f
2. Derive and code EI
3. Run the BO loop
4. Bad kernel: Matérn-1/2 vs. RBF
5. High-dimensional scaling

Rules: fill every ??? before moving on. Written questions carry marks. Use AI to *check* answers, not generate them.

Lab 3 — Section 2: Deriving EI

Setup: the GP gives us $f(\boldsymbol{x}) \sim \mathcal{N}(\mu(\boldsymbol{x}), \sigma^2(\boldsymbol{x}))$ at any candidate.
Let $f^* = \max_i y_i$ be the best observation so far. Define the improvement:

$$I(\boldsymbol{x}) = \max\big(f(\boldsymbol{x}) - f^* - \xi,\ 0\big)$$

Taking the expectation over the GP posterior:

$$\mathrm{EI}(\boldsymbol{x}) = \mathbb{E}[I(\boldsymbol{x})] = \underbrace{(\mu - f^* - \xi)\,\Phi(Z)}_{\text{exploitation}} + \underbrace{\sigma\,\phi(Z)}_{\text{exploration}}$$

where $Z = \dfrac{\mu(\boldsymbol{x}) - f^* - \xi}{\sigma(\boldsymbol{x})}$.

Task

Translate each line of this derivation into Python. The formula is given; your job is to identify the correct variables. Six steps, each one line of code.

Questions to answer

- Which term dominates early in the search? Why?
- What happens when $\xi \to 0$? When ξ is very large?
- Why is EI set to zero where $\sigma \approx 0$?

Lab 3 — Section 4: When BO Fails

4A — Wrong kernel

Himmelblau is a smooth polynomial. The Matérn-$\frac{1}{2}$ kernel assumes the function is *continuous but nowhere differentiable*.

Both runs start from the **same initial sample** (controlled experiment). Only the kernel changes.

What to observe: the Matérn surrogate looks jagged between observations. EI chases surrogate artifacts. Convergence is slower.

Key question: if the function *were* rough, which kernel would be the wrong choice?

4B — High-dimensional scaling

Fix $n = 100$ observations, increase d from 2 to 50. Two measurements:

1. GP fit time (grows as $\mathcal{O}(n^3)$ plus kernel construction)
2. Average nearest-neighbour distance (coverage collapse)

The punchline: as d grows, the GP becomes *simultaneously* slower and less reliable.

This motivates TuRBO, SAASBO, BAXUS — algorithms designed for high-dimensional black-box optimization.

Lab 3 — Section 5: Beyond Vanilla BO

Algorithm	Core idea	Addresses
TuRBO	Local trust region; shrinks/expands with progress	High-d, efficiency
SAASBO	Sparsity prior on length-scales (ARD)	Many irrelevant inputs
BAXUS	Adaptive low-dim subspace embedding	Extreme high-d
Deep Kernel	Neural-network feature map replaces kernel	Complex structure

Every extension to vanilla BO is a **response to a specific failure mode** of the GP surrogate. Understanding *why* BO fails tells you *which* algorithm to reach for.

Lab Q11 (open-ended): You are tuning a neural network with 100 hyperparameters. You believe only 5–10 matter. Which algorithm above would you try first, and why? What evidence would you use to check your assumption?

Closing: The Bigger Picture

What we covered today

- BO on a real engineering problem: PID tuning
- GP surrogate + EI acquisition = principled search
- ARD: the GP reveals which inputs matter
- Failure modes: wrong kernel, high dimension

Where this connects with the previous labs

- Week 2: Gaussians, conditioning, function sampling
- Week 2: GP regression from scratch